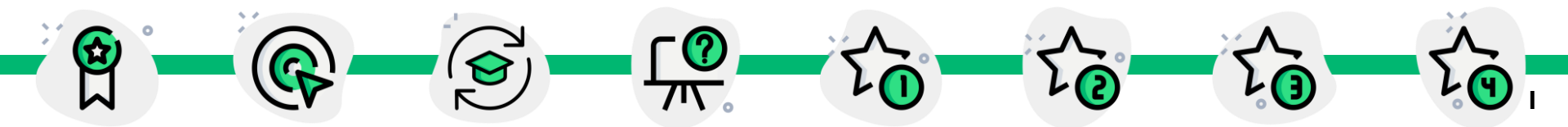


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.04.06

5주차



재귀 알고리즘

05-1 재귀의 기본

05-2 재귀 알고리즘의 분석

05-3 하노이의 탑

05-4 8퀸 문제



05-1 재귀의 기본

재귀의 기본

◆ 재귀적^{recursive}

- 재귀적 정의(recursive definition)에 의해 무한으로 존재하는 자연수를 아래의 두 문장으로 정의할 수 있음
 - ✓ 1은 자연수이다.
 - ✓ 자연수 n 의 바로 다음 수도 자연수이다.
- 재귀를 효과적으로 사용하면 이런 정의 뿐만 아니라 프로그램도 간결하게 할 수 있음



05-1 재귀의 기본

◆ 재귀 개념을 표현한 예



[그림 5-1] 재귀 개념을 표현한 예



05-1 재귀의 기본

◆ 실습 5-1 – 순차곱셈 구하기

- 음이 아닌 정수의 순차곱셈(factorial)을 구하는 프로그램
- factorial 함수는 매개변수 n 에 전달받은 값이 0보다 크면 $n * \text{factorial}(n - 1)$ 을 반환하고, 그렇지 않으면 1을 반환



05-1 재귀의 기본

◆ 실습 5-1 – 순차곱셈 구하기

Chapt05_실습 5-1/factorial.c

```
// 순차곱셈의 결과를 재귀적으로 구하여 출력  
#include <stdio.h>
```

```
/*--- 정수 n의 순차곱셈값을 반환 ---*/
```

```
int factorial(int n)
```

```
{  
    if (n > 0)  
        return n * factorial(n - 1);  
    else  
        return 1;
```

```
}
```

```
int main(void)
```

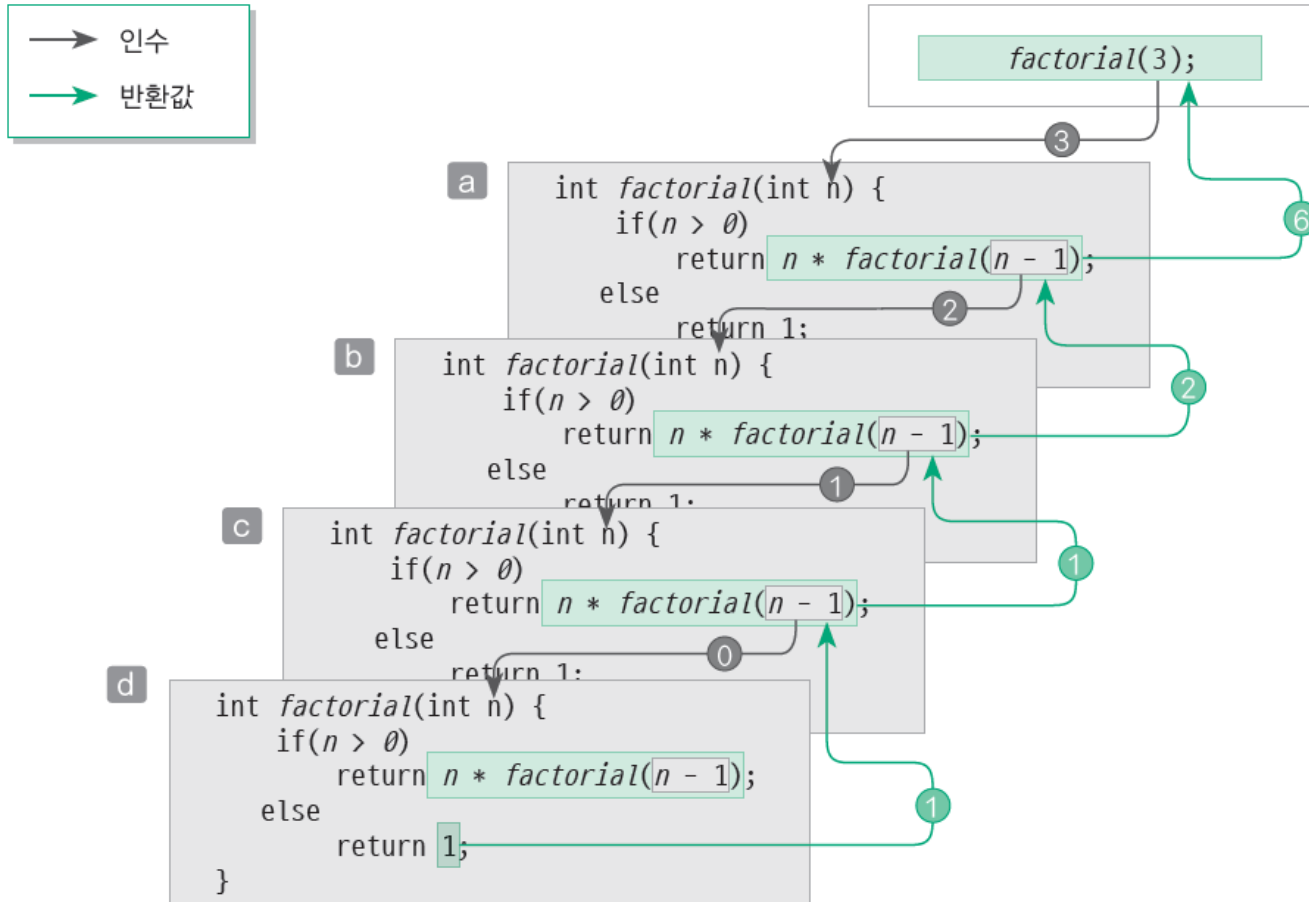
```
{  
    int x;  
    printf("정수를 입력하세요: ");  
    scanf("%d", &x);  
    printf("%d의 순차곱셈값은 %d입니다.\n", x, factorial(x));  
  
    return 0;  
}
```

factorial(3) → 3 * factorial(2)
→ 3 * (2 * factorial(1))
→ 3 * (2 * (1 * factorial(0)))
→ 3 * (2 * (1 * 1)) ← 여기서
factorial(0)은 1을 리턴!
→ 3 * 2 * 1 = 6



05-1 재귀의 기본

◆ 실습 5-1 - 순차 곱셈 구하기



[그림 5-2] 3의 순차곱셈값을 재귀적으로 구하는 과정

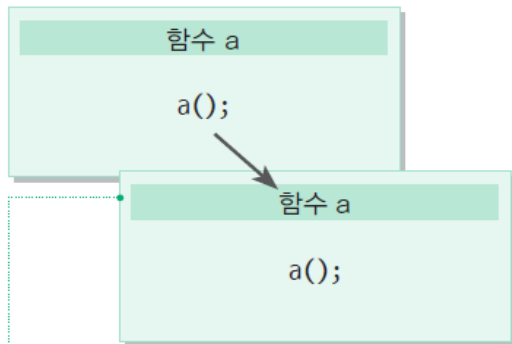


05-1 재귀의 기본

◆ 실습 5-1 – 순차 곱셈 구하기

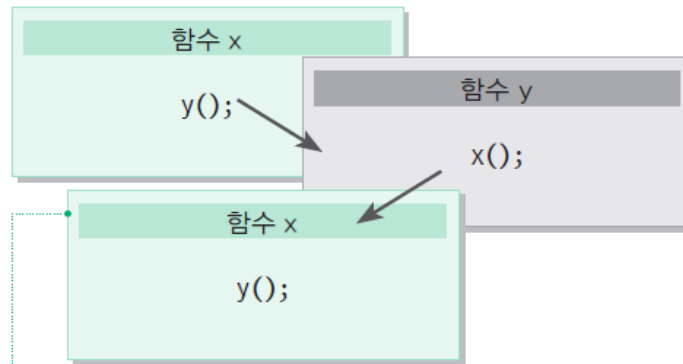
- **직접^{direct} 재귀**
 - ✓ factorial 함수처럼 자신과 같은 함수를 호출
- **간접^{indirect} 재귀**
 - ✓ 함수 x가 함수 y를 호출하고, 다시 함수 y가 함수 x를 호출하는 구조
- **재귀 알고리즘에 알맞은 경우**
 - ✓ 재귀 알고리즘에 알맞은 경우는 '풀어야 할 문제', '계산할 함수', '처리할 데이터 구조'가 재귀로 정의되는 경우

a 직접 재귀



자기 자신과 같은 함수가 호출됩니다.

b 간접 재귀



다른 함수를 통해 자기 자신과 같은 함수가 호출됩니다.

[그림 5-3] 직접 재귀와 간접 재귀



05-1 재귀의 기본

◆ 유클리드 호제법

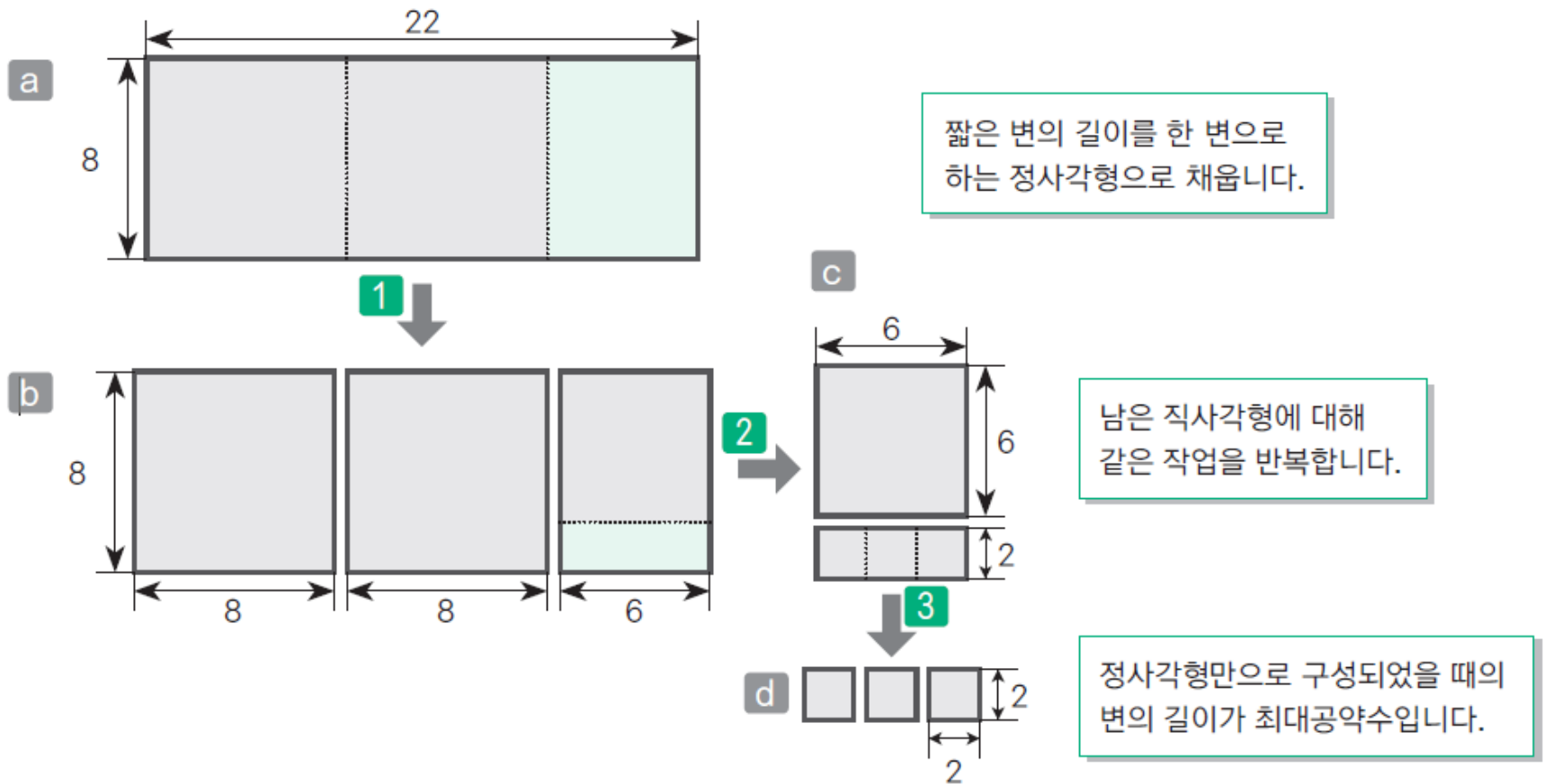
■ 두 정수의 최대공약수를 재귀적으로 구하는 알고리즘

- ✓ 두 정수를 직사각형의 두 변의 길이라고 생각하면 직사각형을 정사각형으로 완전히 채우고 이렇게 만들 수 있는 정사각형의 가장 긴 변의 길이를 구하는 문제로 바꿀 수 있음
- ✓ **1** 22×8 크기의 직사각형에서 짧은 변(8)을 한 변으로 하는 정사각형으로 분할
 - ❖ 8×8 크기의 정사각형 타일 2장과 8×6 크기의 직사각형이 1개 남음
- ✓ **2** 남은 8×6 크기의 직사각형으로 같은 과정을 수행
 - ❖ 6×6 크기의 정사각형이 1개, 6×2 크기의 직사각형이 1개 남음
- ✓ **3** 남은 6×2 크기의 직사각형으로 같은 과정을 수행
 - ❖ 2×2 크기의 정사각형 3개로 나눌 수 있음
- ✓ 여기서 얻은 2가 최대공약수



05-1 재귀의 기본

◆ 유클리드 호제법



[그림 5-4] 22와 8의 최대공약수를 구하는 과정



05-1 재귀의 기본

◆ 실습 5-2 – 유클리드 호제법

- **유클리드 호제법** Euclidean method of mutual division

- ✓ 두 정수 x, y 의 최대공약수를 $\gcd(x, y)$ 로 표기
- ✓ $x = az$ 와 $y = bz$ 를 만족하는 정수 a, b 와 최대의 정수 z 가 존재할 때 z 는 $\gcd(x, y)$
- ✓ 다시 말해 최대공약수는 y 가 0이면 x 이고, y 가 0이 아니면 $\gcd(y, x \% y)$

- **실습 5-2**

- ✓ 유클리드 호제법에 의해 두 정수의 최대공약수를 구하는 프로그램



05-1 재귀의 기본

◆ 실습 5-2 – 유클리드 호제법

Chapt05_실습 5-2/euclid.c

// 유클리드 호제법에 의해 최대공약수를 구하여
출력

```
#include <stdio.h>
```

```
/*--- 정수 x, y의 최대공약수를 반환 ---*/
```

```
int gcd(int x, int y)
```

```
{
```

```
    if (y == 0)
```

```
        return x;
```

```
    else
```

```
        return gcd(y, x % y);
```

```
}
```

```
int main(void)
```

```
{
```

```
    int x, y;
```

```
    puts("두 정수의 최대공약수를 구합니다.");
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf("%d", &x);
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf("%d", &y);
```

```
    printf(" 최대공약수는 %d입니다. \n", gcd(x, y));
```

```
    return 0;
```

```
}
```

실행 결과

두 정수의 최대공약수를 구합니다.

정수를 입력하세요: 22

정수를 입력하세요: 8

최대공약수는 2입니다.



05-2 재귀 알고리즘의 분석

◆ 실습 5-3 – 재귀 알고리즘 분석하기

- recur 함수는 factorial 함수나 gcd 함수와 달리 함수 안에서 재귀 호출을 2회 실행
- 재귀 호출을 여러 회 실행하는 함수를 순수하게(genuinely) 재귀적이라 함
- 실습 5-3의 recur 함수에 매개변수로 4를 전달하면 '1231412'라고 숫자를 한 줄에 한 글자씩 출력
- 복잡한 구조를 가진 재귀 함수는 좀 더 전략적으로 분석해야 함



05-2 재귀 알고리즘의 분석

◆ 실습 5-3 – 재귀 알고리즘 분석하기

Chapt05_실습 5-3/recur.c

// 재귀에 대해 깊이 이해하기 위한 재귀 함수

```
#include <stdio.h>
```

```
/*--- 재귀 함수 recur ---*/
```

```
void recur(int n)
```

```
{
```

```
    if (n > 0) {
```

```
        recur(n - 1);
```

```
        printf("%d\n", n);
```

```
        recur(n - 2);
```

```
    }
```

```
}
```

```
int main(void)
```

```
{
```

```
    int x;
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf("%d", &x);
```

```
    recur(x);
```

```
    return 0;
```

```
}
```



05-2 재귀알고리즘의 분석

◆ 실습 5-3 - 재귀 알고리즘 분석하기

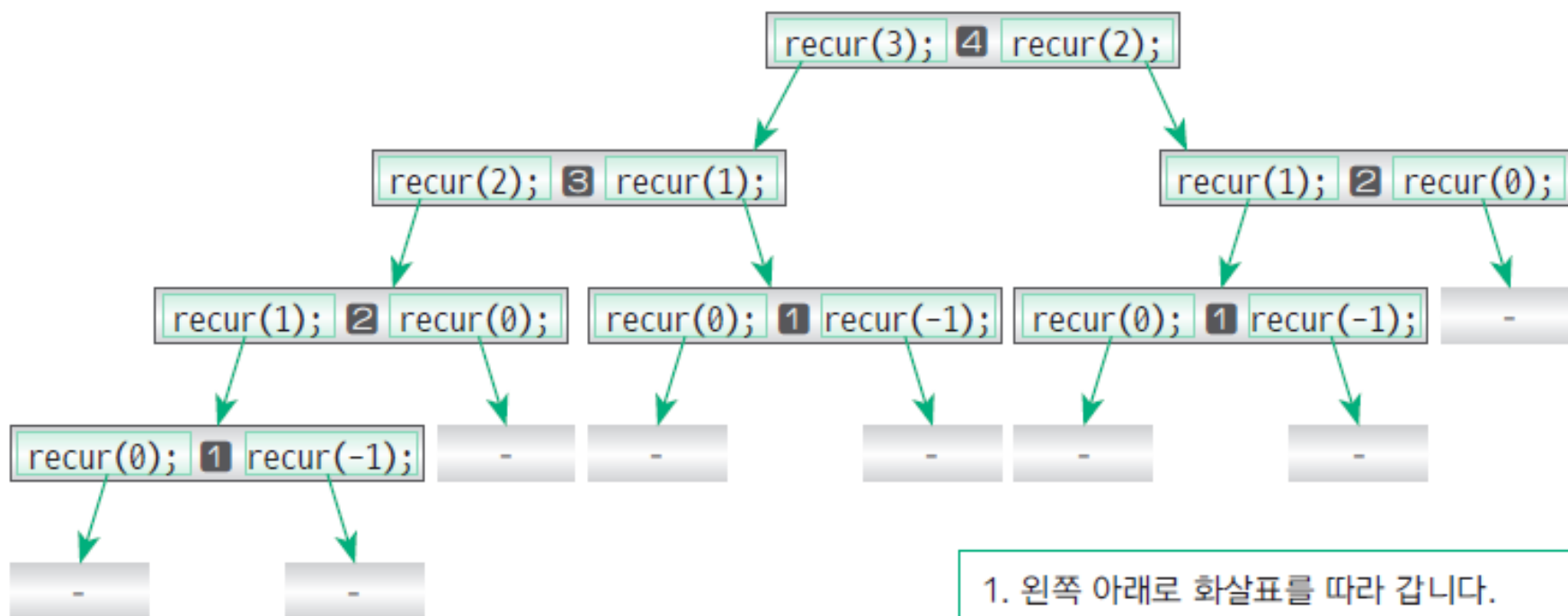
- 하향식 분석 top-down analysis
 - ✓ 각각의 상자는 recur 함수의 동작을 나타냄
 - ✓ 다만 전달받은 값이 0 이하면 recur 함수는 아무 일도 하지 않으므로 빈 상자 표시(상자 안에 '-'를 표기)
 - ✓ '왼쪽 화살표를 따라 하나 아래 상자로 이동하고, 다시 원래의 상자로 돌아오면
 - 안의 값을 출력하고 이어서 오른쪽 화살표를 따라 한 칸 아래 상자로 이동'
 - ✓ 이를 일련의 작업으로 생각하고 작업이 완료되어야 한 칸 위의 상자로 돌아갈 수 있음
 - ✓ 빈 상자에 도달하는 경우 아무것도 하지 않고 돌아감



05-2 재귀 알고리즘의 분석

◆ 실습 5-3 - 재귀 알고리즘 분석하기

- 하향식 분석 top-down analysis



1. 왼쪽 아래로 화살표를 따라 갑니다.
2. 되돌아오면 ■ 안의 값을 출력합니다.
3. 오른쪽 아래로 화살표를 따라 갑니다.

[그림 5-5] recur 함수의 하향식 분석



05-2 재귀 알고리즘의 분석

◆ 실습 5-3 - 재귀 알고리즘 분석하기

■ 하향식 분석 top-down analysis

- ✓ 가장 위쪽에 위치한 상자의 함수 호출부터 시작해 계단식으로 자세히 조사해 가는 분석 기법
- ✓ 가장 위에 있는 상자가 recur(4)의 동작
- ✓ ①의 recur(3)의 호출 이후에 어떤 과정을 거치게 되는지는 왼쪽 아래 화살표를 따라 가면 됨
- ✓ ③의 recur(2)의 호출 이후에 어떤 과정을 거치게 되는지는 오른쪽 아래 화살표를 따라 가면 됨
- ✓ 이 그림 안에는 recur(1), recur(2)의 호출이 여러 번 있음(같은 호출이 여러 번 있음)
- ✓ 꼭대기(top)부터 분석하면 이렇게 같은 함수의 호출이 여러 번 나올 수 있기 때문에 '하향식 분석이 반드시 효율적이다'라고 말할 수는 없음



05-2 재귀 알고리즘의 분석

◆ 실습 5-3 - 재귀 알고리즘 분석하기

- 상향식 분석 bottom-up analysis

- ✓ 하향식 분석과는 대조적으로 아래쪽부터 쌓아 올리며 분석하는 방법
- ✓ recur 함수는 n 이 양수일 때만 실행하므로 먼저 recur(1)을 수행
 - ① recur(0)을 실행 : 출력할 내용이 없음
 - ② 1을 출력
 - ③ recur(-1)을 실행 : 출력할 내용이 없음
- ✓ recur(2)를 수행
 - ① recur(1)을 실행
 - ② 2를 출력
 - ③ recur(0)을 실행 : 출력할 내용이 없음
- ✓ 이러한 과정을 거쳐 recur(4)를 출력



05-2 재귀 알고리즘의 분석

◆ 실습 5-3 - 재귀 알고리즘 분석하기

- 상향식 분석 bottom-up analysis

recur(-1) : 아무것도 하지 않음

recur(0) : 아무것도 하지 않음

.....

recur(1) : recur(0) 1 recur(-1) ⇨ 1

recur(2) : recur(1) 2 recur(0) ⇨ 1 2

recur(3) : recur(2) 3 recur(1) ⇨ 1 2 3 1

recur(4) : recur(3) 4 recur(2) ⇨ 1 2 3 1 4 1 2

[그림 5-6] recur 함수의 상향식 분석



05-2 재귀알고리즘의 분석

◆ 실습 5-4 – 재귀 알고리즘의 비재귀적 표현 살펴보기

- 실습 5-4
 - ✓ 꼬리 재귀를 제거한 프로그램
 - ✓ 함수의 꼬리에서 재귀 호출하는 함수 `recur(n - 2)`라는 말은 '인자로 $n - 2$ 를 전달하여 `recur` 함수를 호출한다'는 의미
 - ✓ 따라서 n 값을 $n - 2$ 로 업데이트하고 함수의 시작 지점으로 돌아가도록 바꿀 수 있음
 - ✓ `recur` 함수는 n 값을 -2만큼 줄인 다음 `goto` 문을 사용해 함수의 시작 지점(레이블(Label) `Top`이 붙은 `if` 문)으로 돌아감



05-2 재귀알고리즘의 분석

◆ 실습 5-4 – 재귀 알고리즘의 비재귀적 표현 살펴보기

Chapt05_실습 5-4/recur_nr1.c

// (말미 재귀 제거)

```
#include <stdio.h>
```

/*--- 말미 재귀를 제거한 recur 함수 ---*/

```
void recur(int n)
```

```
{
```

```
Top:
```

```
    if (n > 0) {
```

```
        recur(n - 1);
```

```
        printf("%d\n", n);
```

```
        n = n - 2;
```

```
        goto Top;
```

```
    }
```

```
}
```

```
int main(void)
```

```
{
```

```
    int x;
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf("%d", &x);
```

```
    recur(x);
```

```
    return 0;
```

```
}
```



05-2 재귀알고리즘의 분석

◆ 실습 5-5 – 재귀 알고리즘의 비재귀적 표현 살펴보기

■ 실습 5-5

- ✓ 재귀를 제거한 프로그램
- ✓ 변수 n 값을 출력하려면 먼저 $\text{recur}(n - 1)$ 을 수행해야 하므로 단순히 n 값을 $n - 1$ 로 업데이트하고 함수의 시작 지점으로 돌아가도록 바꿀 수 없음
- ✓ 현재 n 값을 '잠시' 저장해야 함
 - ❖ 예를 들어 n 이 4인 경우 재귀호출 $\text{recur}(3)$ 의 처리가 완료되지 않으면 n 값인 '4'를 저장해야 함
- ✓ 저장했던 n 을 다시 꺼내 그 값을 출력
- ✓ 변수 n 값을 '잠시' 저장해야 하므로 이런 문제를 잘 해결할 수 있는 데이터 구조인 스택(stack)을 사용



05-2 재귀 알고리즘의 분석

◆ 실습 5-5 – 재귀 알고리즘의 비재귀적 표현 살펴보기

Chapt05_실습 5-5/recur_nr2.c

```
#include <stdio.h>
#include "IntStack.h"
/*--- 재귀 호출을 제거한 recur 함수 ---*/
void recur(int n)
{
    IntStack stk;           // 스택
    Initialize(&stk, 100);

Top:
    if (n > 0) {
        Push(&stk, n);      // n값을 푸시
        n = n - 1;
        goto Top;
    }
    if (!IsEmpty(&stk)) {    // 스택이 비어 있지 않으면
        Pop(&stk, &n);       // 값을 저장했던 n을 팝
        printf("%d\n", n);
        n = n - 2;
        goto Top;
    }
    Terminate(&stk);
}
```

```
int main(void)
{
    int x;

    printf("정수를 입력하세요: ");
    scanf("%d", &x);

    recur(x);

    return 0;
}
```



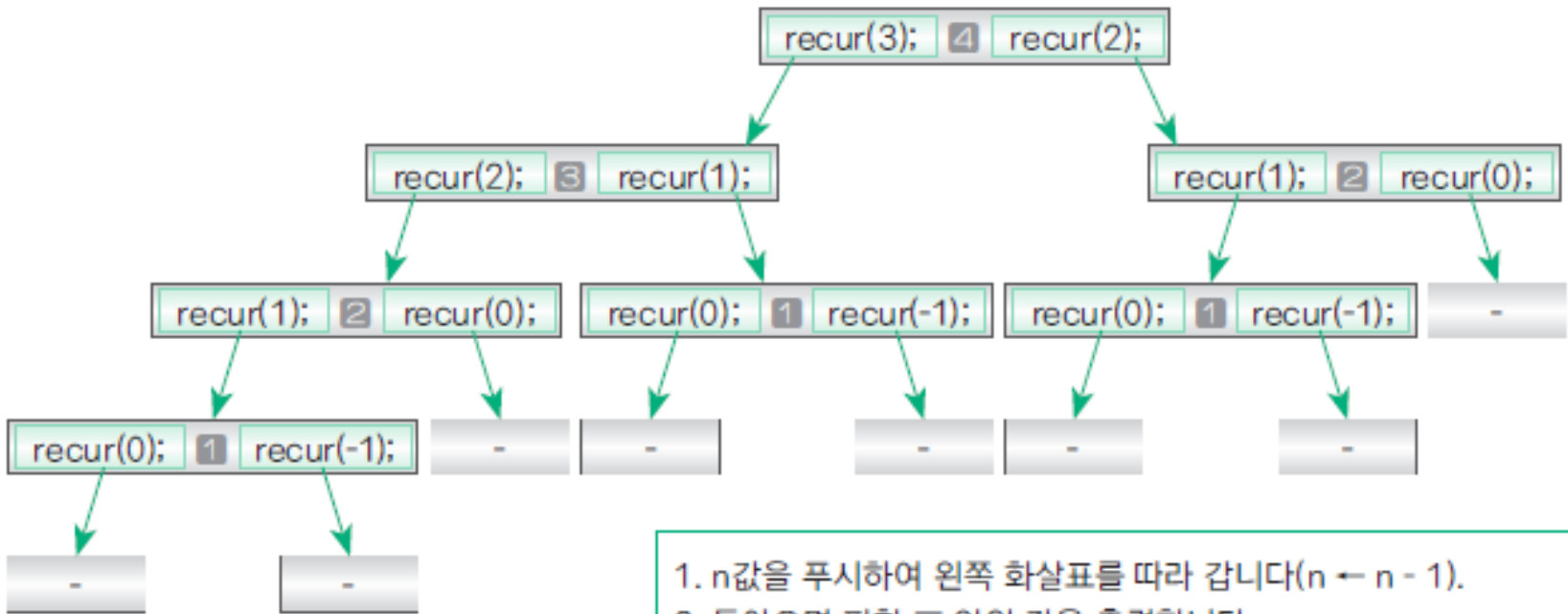
05-2 재귀 알고리즘의 분석

◆ 실습 5-5 – 재귀 알고리즘의 비재귀적 표현 살펴보기

- recur(4)를 호출한 예
 - ✓ 1 4를 스택에 푸시
 - ✓ 2 n값을 하나 줄여 3으로 만듦
 - ✓ 3 goto 문이 실행되어 레이블(Label) Top이 붙은 if 문으로 돌아감
 - ✓ 4 스택에서 팝한 값 1을 n에 꺼내 놓음
 - ✓ 5 n값 1을 출력
 - ✓ 6 n값을 2 줄여 -1로 만듦
 - ✓ 7 goto 문이 실행되어 레이블 Top이 붙은 if 문으로 돌아감



◆ 실습 5-5 - 재귀 알고리즘의 비재귀적 표현 살펴보기

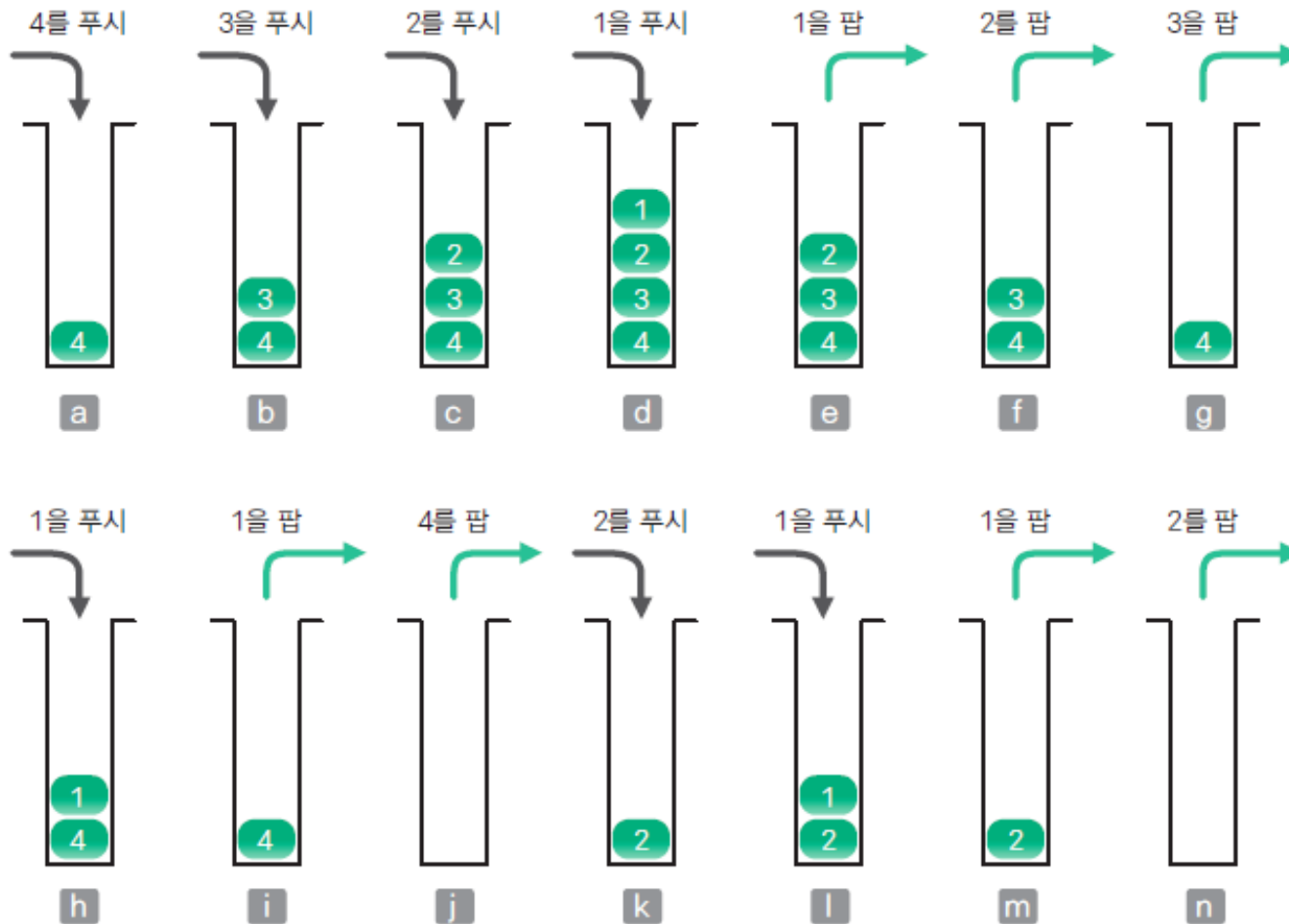


1. n값을 푸시하여 왼쪽 화살표를 따라 갑니다($n \leftarrow n - 1$).
2. 돌아오면 팝한  안의 값을 출력합니다.
3. 오른쪽 화살표를 따라 갑니다($n \leftarrow n - 2$).



05-2 재귀 알고리즘의 분석

◆ 실습 5-5 – 재귀 알고리즘의 비재귀적 표현 살펴보기



[그림 5-7] 실습 5-5의 함수 실행에 따라 변화하는 스택의 모습



05-2 재귀알고리즘의 분석

◆ 메모이제이션 memoization

- 메모이제이션
 - ✓ 재귀를 제거한 프로그램
 - ✓ 메모화' 또는 '메모 기법'이라 불림
 - ✓ recur 함수를 실행하는 과정에서는 여러 번 반복해 같은 계산을 수행하는데 n 값이 커질수록 이처럼 중복되는 계산 횟수는 매우 많아짐
 - ✓ 따라서 같은 계산은 1회만 수행하고 여러 번 계산하지 않도록 수정해야 함



05-2 재귀알고리즘의 분석

◆ 메모이제이션 memoization

- 실습 5-6
 - ✓ 재귀 함수를 메모이제이션으로 구현
 - ✓ memo[] ~ memo[127] 사이에 (NULL을 포함해) 최대 1,024자의 메모를 최대 128개까지 저장
 - ✓ static으로 선언함으로써 모든 문자가 NULL로 초기화
 - ✓ 메모를 이미 한 경우
 - ❖ 1 n에 대해 이미 메모한(즉, 메모가 빈 문자열이 아닌) 경우에는 해당 메모의 내용 memo[n + 1]을 그대로 화면에 출력
 - ✓ 메모를 하지 않은 경우
 - ❖ n이 0보다 클 때
 - 2 원래의 recur와 같은 처리를 수행
 - 3 출력한 문자열을 배열 요소 memo[n + 1]에 메모
 - ❖ n이 0보다 크지 않을 때
 - 4 n은 0 아니면 -1입니다. 빈 문자열 ""을 메모



05-2 재귀알고리즘의 분석

◆ 실습 5-6

Chapt05_실습 5-6/ recur_memo.c

//재귀 함수 recur를 메모이제이션으로 구현

```
#include <stdio.h>
```

```
static char memo[128][1024]; //메모용 문자열 배열
```

/*--- 메모이제이션을 도입한 recur 함수 ---*/

```
void recur(int n){
    if (memo[n+1][0] != '\0')
        printf("%s", memo[n+1]);    //메모 출력
    else{
        if (n > 0){
            recur(n-1);
            printf("%d\n", n);
            recur(n-2);
            sprintf(memo[n+1], "%s%d\n%s", memo[n], n, memo[n-1]);
        }
        else{
            strcpy(memo[n+1], "");
        }
    }
}
```

```
int main(void)
```

```
{
```

```
    int x;
```

```
    printf("정수를 입력하세요: ");
```

```
    scanf("%d", &x);
```

```
    recur(x);
```

```
    return 0;
```

```
}
```

[표 5-1] 함수의 호출 횟수

n	1	2	3	4	5	6	7	8	9	10
오리지널 버전	3	5	9	15	25	41	67	109	177	287
메모이제이션 버전	3	5	7	9	11	13	15	17	19	21